Project Report

For the Course of

Big Data Analytics
(CSBB 422)

Topic: Log File Analysis System

Submitted by

Dilpreet Singh **Roll No. – 231210041**

Ishita Gupta **Roll No. – 231210050**

Madhav Raj **Roll No. – 231210064**

Semester – 7th

Submitted to

Dr. Priten

In the Department of

Computer Science and Engineering



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

NATIONAL INSTITUTE OF TECHNOLOGY, DELHI

Table of Contents

Abstract **3**

1. Introduction **4**

1.1 Background 4
1.2 Problem Statement 4
1.3 Objectives 4
1.4 Scope 4

2. Dataset **5**

2.1 Source 5
2.2 Log Format 5
2.3 Characteristics That Shaped the Design 5

3. System Architecture **6**

3.1 Components 6
3.2 Technology Stack 7
3.3 Design Principles 7

4. Implementation **8**

4.1 Cluster Setup 8
4.2 Ingestion into HDFS 8
4.3 Parsing and ETL (Spark) 9
4.4 Analytics (Spark) 9
4.5 MapReduce Jobs 10
4.6 Anomaly Detection Method 10
4.7 Export, REST API and Dashboard 10
4.8 Pipeline Orchestration 11

5. Results and Analysis **12**

5.1 Data Quality 12
5.2 Error Frequency 12
5.3 Traffic Patterns and Peaks 14
5.4 Most Requested Resources and Most Active Hosts 15
5.5 Anomalies 16
5.6 Validation: Spark versus MapReduce 17
5.7 Running Time of the Full Pipeline 18

6. Testing and Validation **19**

6.1 Issues Found and Resolved 19

7. Team Contribution **20**

8. Conclusion and Future Work **21**

8.1 Conclusion 21
8.2 Future Work 21

References **22**

Abstract

Web servers record every request they receive in access logs. Taken one line at a time these records say little, but analysed together they show how a service is used, when it is under the most load, where it fails, and when something unusual happened. Because such logs quickly grow to millions of lines, they are a natural workload for distributed storage and processing.

This project builds a complete batch pipeline that stores raw server logs in the **Hadoop Distributed File System (HDFS)**, parses and cleans them with **Apache Spark**, computes error frequencies, traffic peaks, top resources and hosts, and detects anomalous hours, and finally presents the results through a **Flask and Chart.js web dashboard**. The same core counts are also implemented as classic **Hadoop MapReduce** jobs on YARN, so that the two engines can be validated against each other and their running times compared.

The system was evaluated on the public NASA Kennedy Space Center web server logs for July and August 1995: **3,461,613 log lines (373 MB)**. All but 12 lines were parsed successfully. Spark and MapReduce produced identical results on every comparison, and Spark completed the comparable jobs 1.4 to 4.0 times faster than MapReduce. The analysis identified 13 July 1995 as the busiest day (134,203 requests), an overall error rate of 0.61%, dominated by "404 Not Found" responses, and two periods with no traffic: an 82-hour gap where the July log file ends early, and a 37-hour server outage in early August. The whole system runs on an ordinary laptop as a containerised Hadoop and Spark cluster started with a single command.

Keywords: big data, log analysis, Hadoop, HDFS, MapReduce, YARN, Apache Spark, PySpark, anomaly detection, Docker, Flask.

1. Introduction

1.1 Background

Every request made to a web server, whether for a page, an image or a script, is written to an access log together with the client's address, the time, the resource requested, the response status and the number of bytes sent. Operations teams use these logs to answer practical questions: how many requests fail and why, at what times the server is busiest, which resources are most popular, and when the behaviour of the system departed from normal.

A busy server produces millions of log lines per month, which is too much to inspect manually and inconvenient to process on a single machine as the volume grows. Distributed frameworks address this by splitting the data into blocks stored across a cluster (HDFS) and processing those blocks in parallel (MapReduce and Spark).

1.2 Problem Statement

The problem assigned for this project was: *"Parse server logs stored in HDFS to find error frequency, traffic peaks, etc."*, using **Hadoop and Spark**.

1.3 Objectives

1. Set up a working Hadoop (HDFS, YARN, MapReduce) and Spark cluster that every team member can run locally.
2. Load a real, publicly available server log dataset into HDFS in a reproducible and verifiable way.
3. Parse the raw log text into a clean, typed dataset, keeping a record of every line that could not be parsed.
4. Compute the error frequency (by status code and over time), traffic volumes and peaks, the most requested resources and most active hosts.
5. Detect anomalous hours: unusual spikes or drops in traffic or error rate, and periods with no data at all.
6. Implement the same core counts as Hadoop MapReduce jobs, verify that both engines agree, and compare their running times.
7. Present the results in an interactive web dashboard backed by a documented REST API.

1.4 Scope

The project is a batch processing system: logs are loaded and analysed on demand rather than as a live stream. It uses access logs only; "errors" therefore means HTTP responses with status 400 or higher (4xx client errors and 5xx server errors).

2. Dataset

2.1 Source

The system is evaluated on the **NASA-HTTP** dataset from the Internet Traffic Archive: two months of access logs from the web server of NASA's Kennedy Space Center in Florida. The files are published as `NASA_access_log_Jul95.gz` and `NASA_access_log_Aug95.gz`.

File	Period	Lines	Size (uncompressed)
NASA_access_log_Jul95	1 Jul – 28 Jul 1995	1,891,715	205,242,368 bytes
NASA_access_log_Aug95	1 Aug – 31 Aug 1995	1,569,898	167,813,770 bytes
Total		**3,461,613**	**373,056,138 bytes**

Table 2.1: The input files.

2.2 Log Format

Each line follows the Common Log Format: the client host, two unused identity fields, the timestamp with its UTC offset, the quoted request line, the HTTP status code and the response size in bytes. All timestamps carry the server's offset of UTC−04:00, and all hourly and daily figures in this report are in that local server time.

```
199.72.81.55 - - [01/Jul/1995:00:00:01 -0400] "GET /history/apollo/ HTTP/1.0" 200 6245
unicomp6.unicomp.net - - [01/Jul/1995:00:00:06 -0400] "GET /shuttle/countdown/
HTTP/1.0" 200 3985
```

2.3 Characteristics That Shaped the Design

- **Hosts are hostnames where possible.** The host field holds a resolved hostname, and an IP address only when no name was available. The analysis therefore reports "top hosts" rather than "top IP addresses".
- **Missing sizes.** The size field is "-" on 33,643 lines. These are stored as 0 bytes with a separate flag.
- **Irregular request lines.** About 3,900 lines (0.11%) have unusual request strings, such as spaces inside the URL or no protocol. Their status and size are valid, so the parser reads the request leniently instead of discarding them. A few lines also contain binary data, so every reader decodes the input tolerantly.
- **Two periods without data.** The July file ends part-way through a line at 28 July 13:32, and the August file starts on 1 August, so there are no records for about three and a half days. Separately, the server was shut down between 1 August 14:52 and 3 August 04:36 because of Hurricane Erin, as noted in the dataset's documentation. Both periods are visible in the results (Section 5.5).

3. System Architecture

The system is organised as a pipeline of independent stages. Each stage reads the output of the previous one from a well-defined location, writes its own output, and can be re-run on its own without affecting the others. Figure 3.1 shows the flow of data.

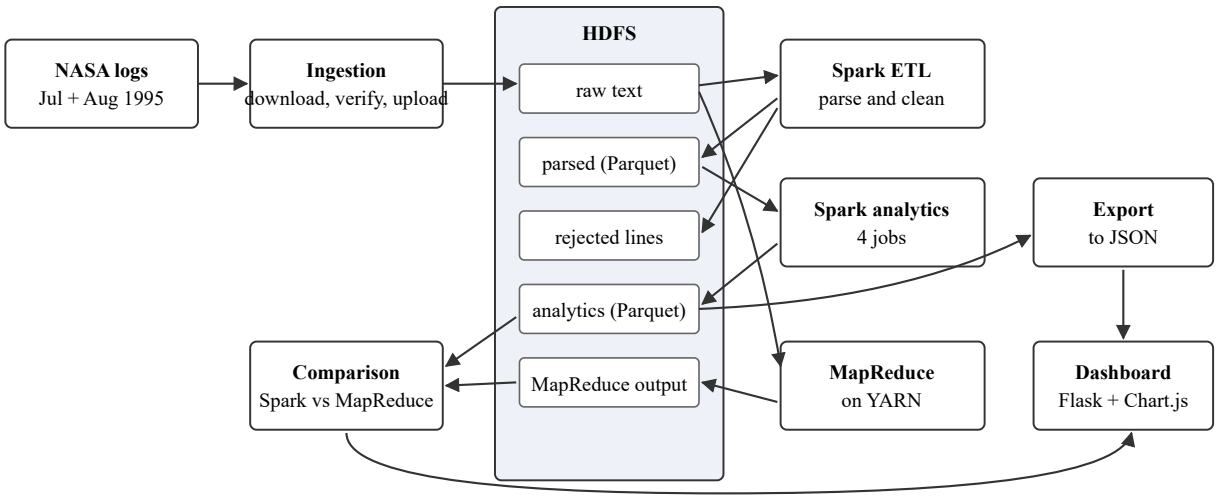


Figure 3.1: Data flow through the system.

3.1 Components

Component	Role
HDFS	Stores the raw logs and every intermediate and final dataset. Files are split into 128 MiB blocks.
YARN	Allocates cluster resources and runs the MapReduce jobs.
Hadoop MapReduce	Classic disk-based processing, used for three validation jobs written in Python through Hadoop Streaming.
Apache Spark (PySpark)	In-memory processing for the ETL and all analytics, running as a standalone cluster that reads and writes HDFS.
Parquet	Compressed columnar file format used for the parsed records and the result tables.
Export (JSON)	Converts the small result tables into JSON files so that the dashboard never needs to access HDFS.
Flask REST API and Chart.js	Serve the results and draw the interactive dashboard in the browser.
Docker Compose	Starts the entire cluster (and the dashboard) on one machine with a single command.

Table 3.1: System components.

3.2 Technology Stack

Technology	Version	Technology	Version
Apache Hadoop	3.4.1	Python	3.10
Apache Spark / PySpark	3.5.3	Java (cluster)	11
Base image	Ubuntu 22.04	Web framework	Flask (served by gunicorn)
Containers	Docker, Docker Compose	Charts	Chart.js
Testing	pytest, ruff	Continuous integration	GitHub Actions

Table 3.2: Technologies and versions.

3.3 Design Principles

- **One parser for both engines.** The regular expressions that define a valid log line exist in a single module used by both Spark and the MapReduce jobs, so both engines read every line in exactly the same way.
- **Logic separated from input and output.** Every analysis is written as a pure function on a DataFrame, which makes it testable on small inputs without a cluster.
- **Configuration instead of hard-coded values.** Paths, thresholds and list sizes come from one configuration file. The same code runs on a laptop against the local file system or on the cluster against HDFS.
- **Fixed data contracts.** Every output table has a defined set of columns and types. Jobs check their output against it before writing.
- **Repeatable stages.** Re-running a stage replaces its own output and nothing else.

4. Implementation

4.1 Cluster Setup

All services run from one Docker image built from Ubuntu 22.04 and the official Hadoop and Spark releases, whose checksums are verified during the build. A single image was chosen because the team used both Windows laptops (x86-64) and an Apple Silicon laptop (ARM64), and the same build works natively on both. The default *lite* profile fits in about 6 GB of memory:

Service	Purpose	Web interface
namenode	HDFS metadata	port 9870
datanode	HDFS data blocks	–
resourcemanager	YARN scheduler	port 8088
nodemanager	Runs MapReduce tasks	–
spark-master, spark-worker	Spark standalone cluster (one worker, 2 cores, 1 GB)	port 8080
client	Submits jobs and runs the pipeline	–
dashboard	Flask web application	port 5000

Table 4.1: Services of the containerised cluster (lite profile).

Health checks make each service wait for the ones it depends on, so that no job starts before HDFS is ready. A smoke test confirms the cluster works end to end: writing to HDFS, running a Spark job, running the bundled MapReduce word-count example, and running a Python mapper through Hadoop Streaming.

4.2 Ingestion into HDFS

The ingestion stage downloads both files, resuming interrupted transfers, and verifies each against a recorded SHA-256 checksum. It then decompresses them before upload, because gzip files cannot be split and would force a single task to read each file. The files are written to `/logs/raw/nasa/month=1995-07/` and `month=1995-08/`, and their sizes in HDFS are checked against the local copies. The upload is repeatable: files already present with the same size are skipped.

Measure	Value
Data stored	373,056,138 bytes in 2 files
HDFS blocks	4 (block size 128 MiB)
Replication	1 (lite profile, single DataNode)
File-system check (fsck)	HEALTHY

Table 4.2: Result of loading the dataset into HDFS.

4.3 Parsing and ETL (Spark)

The ETL job reads the raw text from HDFS and applies the Common Log Format regular expression with Spark's native `regexp_extract`, which avoids the overhead of Python functions. Each line is either parsed into typed fields or rejected with a reason. The checks run in a fixed order:

Reason	Meaning
NO_MATCH	The line is not in Common Log Format at all
BAD_TIMESTAMP	The timestamp is not a real date and time
BAD_STATUS	The status code is outside 100–599
BAD_REQUEST	The request has no method and resource

Table 4.3: Rejection reasons.

For each parsed line the job derives the timestamp, date and hour in server time, the status class (2xx to 5xx), an error flag, the byte count and the resource type (file extension). It then writes three outputs:

- the parsed records as Parquet, partitioned by date;
- the rejected lines with their reason and source file;
- a one-row data-quality report.

The job fails if the number of parsed and rejected lines does not add up to the number of input lines.

4.4 Analytics (Spark)

Four independent jobs read the parsed data and write 14 result tables:

Job	Result tables
Errors	requests per status code with share of total; requests, 4xx, 5xx and error rate per hour; top resources for each error status; hosts receiving the most errors
Traffic	requests, bytes and unique hosts per hour and per day; average requests by hour of day and by day of week; the ten busiest minutes, hours and days
Top-N	top 20 resources by requests and by bytes; top 20 hosts; requests and bytes by resource type
Anomalies	hours flagged as spikes, drops or gaps (Section 4.6)

Table 4.4: Analytics jobs and their outputs.

Rankings break ties by the key in alphabetical order, the same rule the MapReduce jobs use, so that ranked lists from the two engines can be compared row by row. Unique host counts are exact.

4.5 MapReduce Jobs

Three counts are also computed with Hadoop MapReduce, written in Python and run on YARN through Hadoop Streaming. The mappers use the same parsing module as Spark, which is packaged and shipped to the cluster nodes with each job. Malformed lines are skipped and counted with a Hadoop counter.

Job	Mapper emits	Reducer
status_count	(status code, 1)	sum per status code (also used as combiner)
hourly_traffic	(date and hour, 1)	sum per hour (also used as combiner)
top_resources	stage 1: (resource, 1); stage 2: each mapper's local top 20	stage 1: sum per resource; stage 2: a single reducer merges the global top 20

Table 4.5: MapReduce jobs.

4.6 Anomaly Detection Method

Web traffic follows a strong daily cycle, so an hour is compared with the **same hour of day on the previous 7 days** rather than with its neighbours. The method is applied to two metrics, requests per hour and error rate per hour:

1. A complete hourly calendar is built from the first to the last hour of data, so an hour with no requests appears as zero instead of being missing. Such an hour is marked as a **gap**.
2. For every other hour, the mean and standard deviation of the same hour on the previous 7 days are computed. Gap hours are left out of this baseline, so an outage does not distort the following week.
3. The hour's z-score is $z = (value - mean) / standard\ deviation$. A z-score above +3 is a **spike**, and below −3 a **drop**.
4. An hour is not scored when it has fewer than 3 baseline values (the first days of the data). Its error rate is not scored when it has fewer than 50 requests.

All thresholds are set in the configuration file.

4.7 Export, REST API and Dashboard

The export stage converts every result table into a JSON file with formatted timestamps and writes a manifest file last, so that an incomplete export is never mistaken for a complete one. It produced 18 files: 16 tables and 2 run reports.

A Flask application reads these files through a single data-access layer and serves them through a REST API. Endpoints cover the summary, errors, traffic, top resources and hosts, anomalies, data quality and the engine benchmark, with validation of query parameters and JSON error responses.

The browser dashboard is drawn with Chart.js and has ten pages that follow the pipeline:

- Overview and Architecture

- Ingestion and Parsing
- Errors, Traffic, Top N and Anomalies
- Engine comparison
- Export and API

It also includes a built-in user guide, and supports light and dark themes.

4.8 Pipeline Orchestration

A command-line runner executes the whole pipeline, or any single stage, with the same command on Windows, macOS and Linux:

```
python scripts/pipeline.py all --env cluster
python scripts/pipeline.py analytics --only errors,traffic --env cluster
```

The stages run in order: ingest, ETL, analytics, MapReduce, comparison and export. The runner records each stage's running time, and stops at the first failure.

5. Results and Analysis

All results in this section come from a complete run of the pipeline on the full dataset on the containerised cluster (lite profile, Windows 11 laptop).

3,461,601 requests analysed	**137,978** unique hosts	**65.5 GB** data served (65,524,314,915 bytes)
0.61% error rate (status ≥ 400)	**14,926** requests in the busiest hour	**58 of 62** days with traffic

5.1 Data Quality

Measure	Value
Input lines	3,461,613
Parsed successfully	3,461,601 (99.9997%)
Rejected: BAD_REQUEST (binary data in the request line)	11
Rejected: NO_MATCH (the truncated last line of the July file)	1
Time range	1 Jul 1995 00:00:01 – 31 Aug 1995 23:59:53
Hours with traffic	1,369 of 1,488

Table 5.1: Data-quality report produced by the ETL.

The MapReduce jobs skipped exactly the same 12 lines, which confirms that both engines apply the same definition of a valid line.

5.2 Error Frequency

Successful responses (200) make up 89.57% of all requests, and redirects and "not modified" responses (302, 304) a further 9.82%. Errors are rare: 21,236 requests, or 0.61%. Almost all of them are 404 "Not Found" responses, which point to broken links. Server errors (500 and 501) occurred only 106 times in two months.

Status	Meaning	Requests	Share
200	OK	3,100,522	89.57%
302	Found (redirect)	73,070	2.11%
304	Not modified (cached copy used)	266,773	7.71%
400	Bad request	4	0.0001%
403	Forbidden	225	0.0065%
404	Not found	20,901	0.60%
500	Internal server error	65	0.0019%
501	Not implemented	41	0.0012%

Table 5.2: Requests by HTTP status code.

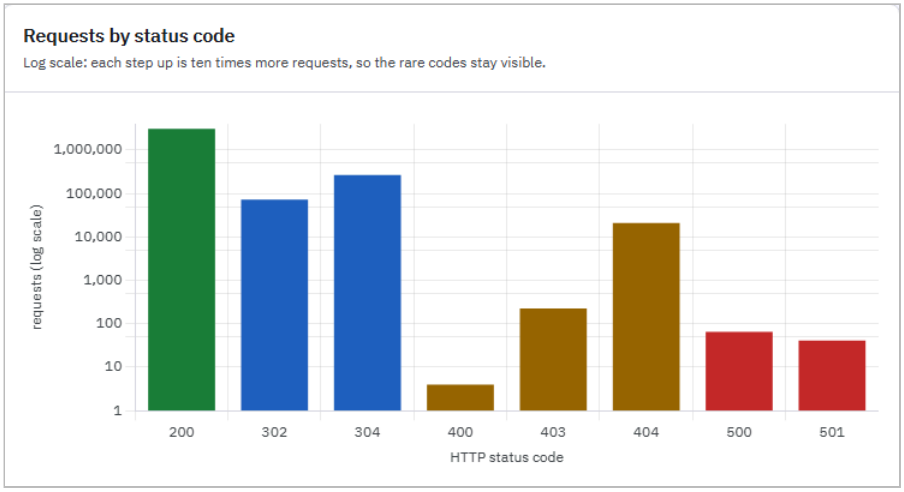


Figure 5.1: Requests by status code (logarithmic scale, so that the rare codes remain visible).



Figure 5.2: Daily error rate for 4xx and 5xx responses. Breaks in the lines are the two periods without data.

#	Resource returning 404 (broken link)	Requests
1	`/pub/winvn/readme.txt`	2,004
2	`/pub/winvn/release.txt`	1,732
3	`/shuttle/missions/STS-69/mission-STS-69.html`	683

Table 5.3: The most frequent broken links. The first two alone account for 18% of all 404 responses.

#	Host receiving the most error responses	Errors
1	`hoohoo.ncsa.uiuc.edu`	251
2	`piweba3y.prodigy.com`	157
3	`jbiagioni.npt.nuwc.navy.mil`	132
4	`piweba1y.prodigy.com`	114
5	`www-d4.proxy.aol.com`	91

Table 5.4: Hosts that received the most error responses.

5.3 Traffic Patterns and Peaks

The server handled an average of 59,683 requests per day over the 58 days with traffic. Figure 5.3 shows the daily totals. One day stands out: **13 July 1995**, with 134,203 requests, more than twice a normal day. It is also the busiest day, hour and minute of the dataset, and it coincides with the launch of Space Shuttle mission STS-70 from the Kennedy Space Center that morning.



Figure 5.3: Requests per day. The grey band has no log data (end of the July file); the red band is the server outage.

Rank	Hour starting (server time, UTC−04:00)	Requests
1	13 Jul 1995, 09:00	14,926
2	13 Jul 1995, 08:00	11,567
3	13 Jul 1995, 10:00	10,214
4	13 Jul 1995, 11:00	9,748
5	13 Jul 1995, 12:00	8,533
6	5 Jul 1995, 14:00	8,034

Table 5.5: The busiest hours. The busiest day was 13 July (134,203 requests), followed by 6 July (100,960); the busiest single minute was 13 July 09:49, with 405 requests.

The daily and weekly rhythms are clear (Figure 5.4). Traffic is lowest in the early morning, around 1,000 requests per hour at 04:00–05:00, and peaks in the afternoon, with an average of 3,977 requests at 15:00. Weekdays are far busier than weekends: Thursday averages 74,193 requests, while Saturday and Sunday average about 39,700, roughly half.

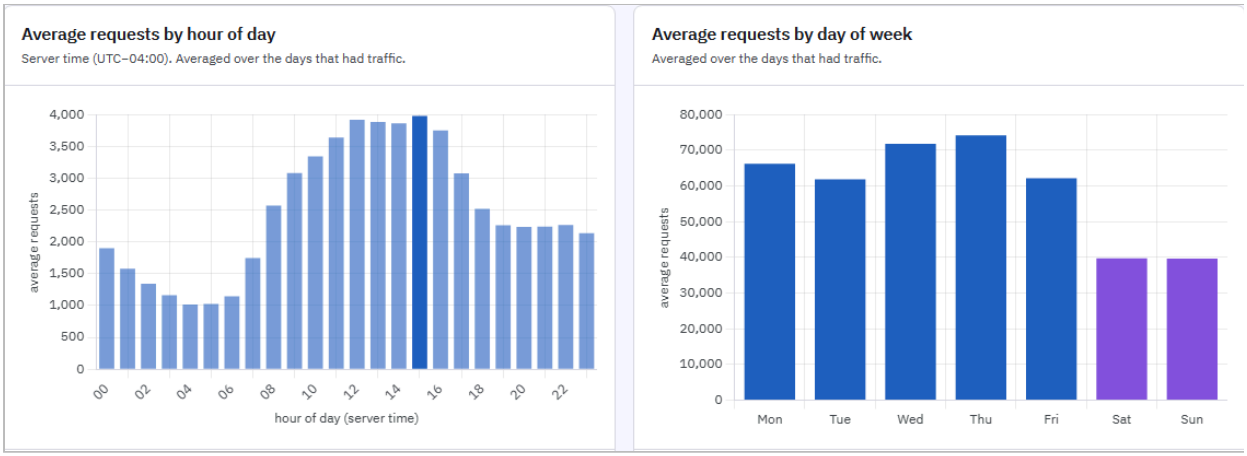


Figure 5.4: Average requests by hour of day and by day of week, over the days with traffic.

Day	Mon	Tue	Wed	Thu	Fri	Sat	Sun
Total requests	529,954	556,589	574,547	667,733	497,456	318,046	317,276
Average per day	66,244	61,843	71,818	74,193	62,182	39,756	39,660

Table 5.6: Requests by day of week.

5.4 Most Requested Resources and Most Active Hosts

The most requested files are small images embedded in almost every page, led by the NASA logo with 208,798 requests. GIF images account for 57.5% of all requests and HTML pages for 21.7%. The busiest hosts are proxy servers of online service providers such as Prodigy and AOL, which forward the requests of many individual users.

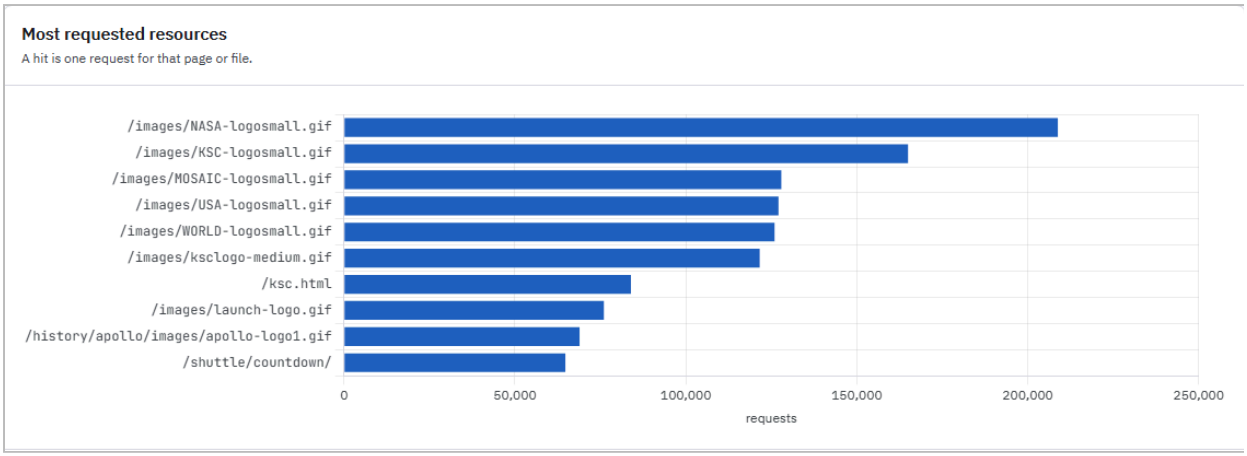


Figure 5.5: The ten most requested resources.

#	Host	Requests	Type	Requests	Share
1	piweba3y.prodigy.com	21,988	gif	1,989,672	57.5%
2	piweba4y.prodigy.com	16,437	html	751,442	21.7%
3	piweba1y.prodigy.com	12,825	directory	232,327	6.7%
4	edams.ksc.nasa.gov	11,964	xbm	110,341	3.2%
5	163.206.89.4	9,697	pl	79,962	2.3%
6	news.ti.com	8,161	jpg	79,765	2.3%
7	www-d1.proxy.aol.com	8,047	none	76,510	2.2%
8	alyssa.prodigy.com	8,037	txt	51,555	1.5%
9	siltb10.orl.mmc.com	7,573	other (463 types)	90,027	2.6%
10	www-a2.proxy.aol.com	7,516			

Table 5.7: The ten most active hosts (left) and requests by resource type (right).

5.5 Anomalies

The anomaly job flagged 268 hours in total:

Kind	Requests per hour	Error rate
Gap (no requests)	119	–
Spike ($z > +3$)	34	88
Drop ($z < -3$)	23	4

Table 5.8: Flagged hours by metric and kind.

The gap hours form exactly the two periods identified in the dataset (Section 2.3):

Period	Length	Cause
28 Jul 14:00 – 31 Jul 23:00	82 hours	No log data: the July file ends early
1 Aug 15:00 – 3 Aug 03:00	37 hours	Server outage (Hurricane Erin)

Table 5.9: Periods without traffic.

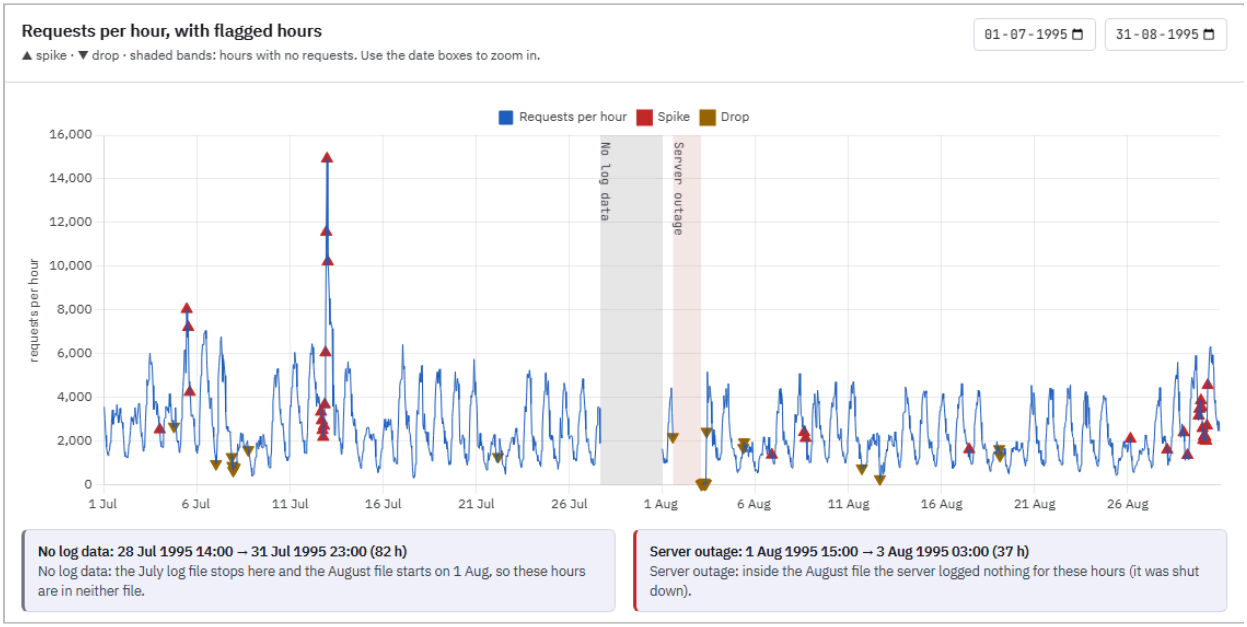


Figure 5.6: Requests per hour with flagged spikes (red, pointing up) and drops (brown, pointing down).

Among the traffic anomalies, the clearest is the 10-hour surge on the morning of 13 July (z-scores up to 6.5), matching the peak described in Section 5.3. The method also captured the server's recovery after the outage: on 3 August between 04:00 and 11:00 traffic was far below normal (for example, 57 requests at 10:00 against a usual 3,567).

The error-rate spikes need care when interpreted. Because the usual error rate is very small (about 0.5%), a small absolute rise, such as to 1% in one hour, already produces a z-score above 3. Most of the 88 error-rate spikes are of this kind. The traffic anomalies are a more reliable signal; for the error rate, a higher threshold or a minimum absolute increase would make the alerts more selective (Section 8.2).

5.6 Validation: Spark versus MapReduce

The three MapReduce jobs were run on YARN on the same raw data, and their output was compared key by key with the corresponding Spark tables. All three results match exactly:

Job	Compared with Spark table	Rows	Result
status_count	status_counts	8	identical
hourly_traffic	traffic_hourly	1,369	identical
top_resources	top_resources_by_hits (top 20)	20	identical, in the same order

Table 5.10: Result of the engine comparison.

Each job was then timed three times on each engine, including start-up, and the median was kept:

Job	Spark	MapReduce	MapReduce / Spark
status_count	48.1 s	97.9 s	2.0×
hourly_traffic	77.2 s	107.5 s	1.4×
top_resources	41.1 s	163.3 s	4.0×

Table 5.11: Median running times (lite profile, single worker).

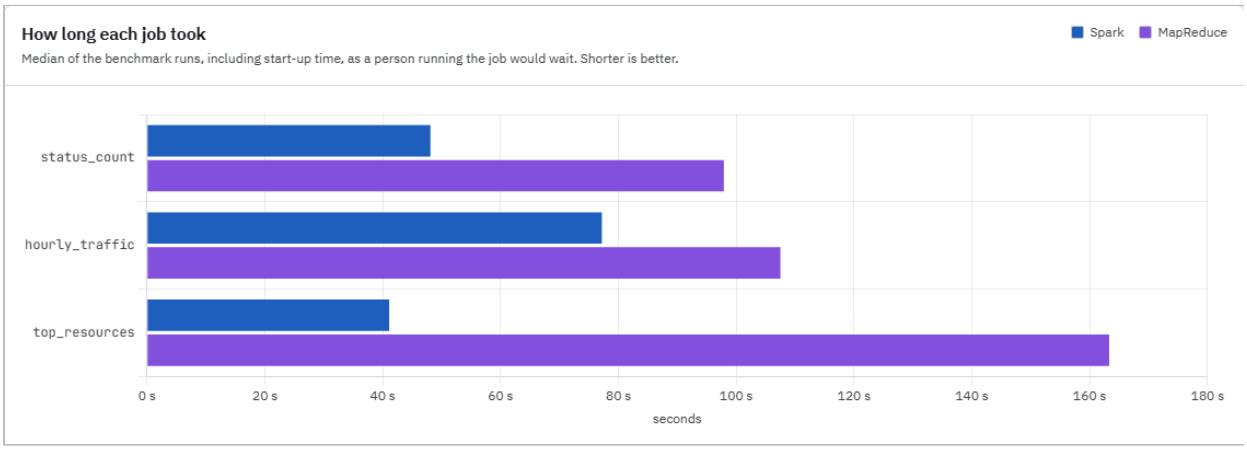


Figure 5.7: Running time of each job on both engines.

Spark was faster on every job. The largest difference is on top_resources, which MapReduce performs as two chained jobs that each write their intermediate results to disk, whereas Spark keeps them in memory. The Spark timings include running the complete analytics job behind each table (for example, the traffic job computes five tables), so the comparison favours MapReduce, if anything.

5.7 Running Time of the Full Pipeline

Stage	Time
Ingestion into HDFS	48 s
Spark ETL (3.46 million lines)	226 s
Spark analytics (4 jobs)	about 1 min each
MapReduce (3 jobs)	339 s
Comparison and benchmark (3 runs of each job on both engines)	1,725 s
Export	about 40 s
Complete pipeline	**2,669 s (about 45 min)**

Table 5.12: Running time of each stage in a complete run. Without the benchmark, the pipeline finishes in about 16 minutes.

6. Testing and Validation

The project was tested at several levels, and the tests run automatically on every push to the repository through GitHub Actions (linting with ruff and the full pytest suite).

Level	What is tested
Parser	Valid lines and every kind of malformed line (missing fields, odd requests, impossible dates, bad status codes).
Parser parity	Spark and Python parsers give identical results on the test fixtures and a 2,024-line real sample.
Spark analytics	Each analysis function on small inputs with known answers, including ties and empty periods.
Anomaly detection	Synthetic traffic with a planted gap and two spikes: exactly these are flagged, nothing else.
Integration	ETL and all analytics on the real sample, checked against contracts, a Python count and MapReduce.
Export and API	Every REST endpoint, parameter validation and error responses, on real exported data.
Cluster	Smoke test of HDFS, Spark, MapReduce and Hadoop Streaming, then full runs on the dataset.

Table 6.1: Test levels.

In the cluster's client container, 132 tests pass, covering the parser, Spark jobs, MapReduce jobs and export. The dashboard's 68 API tests also pass. On the full dataset, the results were checked against values measured independently: the line and byte counts of the input files, the 12 rejected lines, the 1,369 hours with traffic, and the two known data gaps.

6.1 Issues Found and Resolved

- **Spark's date parser.** By default, Spark 3 raises an error on impossible dates such as "32/Jul" instead of returning an empty value. The parser policy was set so that such lines are rejected cleanly.
- **Time zones.** The container image initially lacked the operating system's time-zone database, so Python ignored the server time zone and timestamps read into Python were four hours off. Installing the database fixed it, and a test now fails clearly if the zone is not applied.
- **Resource deadlock in the benchmark.** The comparison script's open Spark session held all cores of the single worker, so the benchmark jobs waited indefinitely. The session is now released first.
- **Clock changes and disk space.** Durations now use a monotonic clock, after the virtual machine's clock moved backwards in one run; and the image is built once for all services, after parallel builds filled a laptop's disk.

7. Team Contribution

The work was divided into three tracks of similar size, with data contracts agreed at the start so that the tracks could be developed in parallel. Each member worked on feature branches, and the work was integrated through the shared GitHub repository.

Member	Track	Work
Dilpreet Singh 231210041	Platform, ingestion and MapReduce	Docker image and Hadoop/Spark cluster configuration with lite and full profiles; dataset download with checksum verification and loading into HDFS; the three MapReduce jobs; the pipeline runner; the Spark versus MapReduce comparison.
Ishita Gupta 231210050	Export, dashboard and delivery	Export of the result tables to JSON; the Flask REST API; the ten-page Chart.js dashboard and its user guide; the dashboard container; the demonstration script and documentation.
Madhav Raj 231210064	Parsing, Spark processing and testing	The shared log parser and its Spark version; common Spark utilities and table contracts; the Spark ETL job; the errors, traffic, top-N and anomaly-detection jobs; the test suite and continuous integration.

Table 7.1: Division of work.

8. Conclusion and Future Work

8.1 Conclusion

The project delivered a complete, working log analysis system on Hadoop and Spark. Raw server logs are stored in HDFS, parsed and cleaned with Spark, analysed for error frequency, traffic peaks, top resources and hosts and anomalies, cross-checked against Hadoop MapReduce, and presented through an interactive dashboard. On 3.46 million real log lines the system:

- parsed all but 12 lines;
- produced results identical to MapReduce on every comparison, while Spark ran 1.4 to 4.0 times faster;
- surfaced meaningful findings: the 13 July launch-day peak, the daily and weekly rhythms of traffic, the broken links behind most errors, and both periods without data.

The entire system runs on a single laptop and is started with one command.

8.2 Future Work

- **More selective error-rate alerts**, using a separate threshold for the error rate or a minimum absolute increase, as discussed in Section 5.5.
- **Real-time processing** with Spark Structured Streaming, so that anomalies are detected as logs arrive rather than in batches.
- **Larger clusters**: running the full profile (two DataNodes and two Spark workers, replication 2) and multi-node deployments to study scaling.
- **Further analyses**, such as user sessions, geographic distribution of hosts and per-resource error trends.

References

1. Internet Traffic Archive, "NASA-HTTP: Two months of HTTP logs from the KSC-NASA WWW server," https://ita.ee.lbl.gov/html/contrib/NASA-HTTP.html
2. The Apache Software Foundation, "Apache Hadoop 3.4.1 Documentation," https://hadoop.apache.org/docs/r3.4.1/
3. The Apache Software Foundation, "Hadoop Streaming," https://hadoop.apache.org/docs/r3.4.1/hadoop-streaming/HadoopStreaming.html
4. The Apache Software Foundation, "Apache Spark 3.5.3 Documentation," https://spark.apache.org/docs/3.5.3/
5. The Apache Software Foundation, "Apache Parquet," https://parquet.apache.org/
6. J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.
7. M. Zaharia et al., "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing," in *Proc. 9th USENIX NSDI*, 2012.
8. Pallets Projects, "Flask Documentation," https://flask.palletsprojects.com/
9. Chart.js, "Chart.js Documentation," https://www.chartjs.org/docs/
10. Project repository: https://github.com/Madhav064/Log-File-Analysis-System